

Lab 1: Model Validation with Cross-Validation

Objective

The goal of this lab is to understand how to validate a machine learning model using cross-validation.

Prerequisites

- Basic understanding of Python
- Basic understanding of machine learning concepts
- Python libraries: numpy, pandas, scikit-learn, matplotlib (Ensure these are installed in your environment)

Dataset

We'll use the Iris dataset, which is a classic dataset for classification tasks.

Steps

1. **Load the Dataset**
2. **Split the Dataset**
3. **Train a Model**
4. **Evaluate the Model using Cross-Validation**
5. **Visualise the Results**

Solution:

Step 1: Load the Dataset

```
import numpy as np

import pandas as pd

from sklearn.datasets import load_iris

# Load the Iris dataset

iris = load_iris()

X = iris.data

y = iris.target

# Create a DataFrame for Visualisation

df = pd.DataFrame(X, columns=iris.feature_names)

df['target'] = y

print(df.head())
```

Step 2: Split the Dataset

```
from sklearn.model_selection import train_test_split

# Split the dataset into training and testing sets

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
```

Step 3: Train a Model

We will train a simple machine learning model, such as a K-Nearest Neighbors (KNN) classifier.

```
from sklearn.neighbors import KNeighborsClassifier
```

```
# Create and train the KNN classifier

knn = KNeighborsClassifier(n_neighbors=3)

knn.fit(X_train, y_train)
```

Step 4: Evaluate the Model using Cross-Validation

```
from sklearn.model_selection import cross_val_score

# Perform 5-fold cross-validation

cv_scores = cross_val_score(knn, X, y, cv=5)

print(f"Cross-validation scores: {cv_scores}")

print(f"Mean cross-validation score: {np.mean(cv_scores)}")
```

Step 5: Visualise the Results

```
import matplotlib.pyplot as plt

# Plot the cross-validation scores

plt.figure(figsize=(10, 6))

plt.plot(range(1, 6), cv_scores, marker='o', linestyle='--')

plt.xlabel('Fold')

plt.ylabel('Accuracy')

plt.title('Cross-Validation Scores')

plt.ylim(0.8, 1.0)

plt.show()
```

Conclusion

In this lab, we covered the basics of model validation using cross-validation. We loaded the Iris dataset, split it into training and testing sets, trained a KNN classifier, and evaluated it using 5-fold cross-validation. Finally, we visualized the cross-validation scores to understand the model's performance better.

Lab: Evaluating a Classification Model with a Confusion Matrix

Objective

The goal of this lab is to understand how to use a confusion matrix to evaluate the performance of a classification model.

Prerequisites

- Basic understanding of Python
- Basic understanding of machine learning concepts
- Python libraries: numpy, pandas, scikit-learn, matplotlib (Ensure these are installed in your environment)

Dataset

We'll use the Iris dataset, which is a classic dataset for classification tasks.

Steps

1. **Load the Dataset**
2. **Split the Dataset**
3. **Train a Model**
4. **Make Predictions**
5. **Compute the Confusion Matrix**
6. **Visualise the Confusion Matrix**

Solution:

Step 1: Load the Dataset

```
import numpy as np

import pandas as pd

from sklearn.datasets import load_iris

# Load the Iris dataset

iris = load_iris()

X = iris.data

y = iris.target

# Create a DataFrame for better visualization

df = pd.DataFrame(X, columns=iris.feature_names)

df['target'] = y

print(df.head())
```

Step 2: Split the Dataset

```
from sklearn.model_selection import train_test_split

# Split the dataset into training and testing sets

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
```

Step 3: Train a Model

```
from sklearn.neighbors import KNeighborsClassifier

# Create and train the KNN classifier
```

```
knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X_train, y_train)
```

Step 4: Make Predictions

```
# Make predictions on the test set
y_pred = knn.predict(X_test)
```

Step 5: Compute the Confusion Matrix

We will compute the confusion matrix to evaluate the model's performance.

```
from sklearn.metrics import confusion_matrix

# Compute the confusion matrix
cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:")
print(cm)
```

Step 6: Visualise the Confusion Matrix

```
import matplotlib.pyplot as plt
import seaborn as sns

# Plot the confusion matrix as a heatmap
plt.figure(figsize=(8, 6))

sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=iris.target_names,
            yticklabels=iris.target_names)

plt.xlabel('Predicted Labels')
```

```
plt.ylabel('True Labels')  
plt.title('Confusion Matrix')  
plt.show()
```

Conclusion

In this lab, we covered the basics of using a confusion matrix to evaluate a classification model. We loaded the Iris dataset, split it into training and testing sets, trained a KNN classifier, made predictions, computed the confusion matrix, and visualized it using a heatmap.